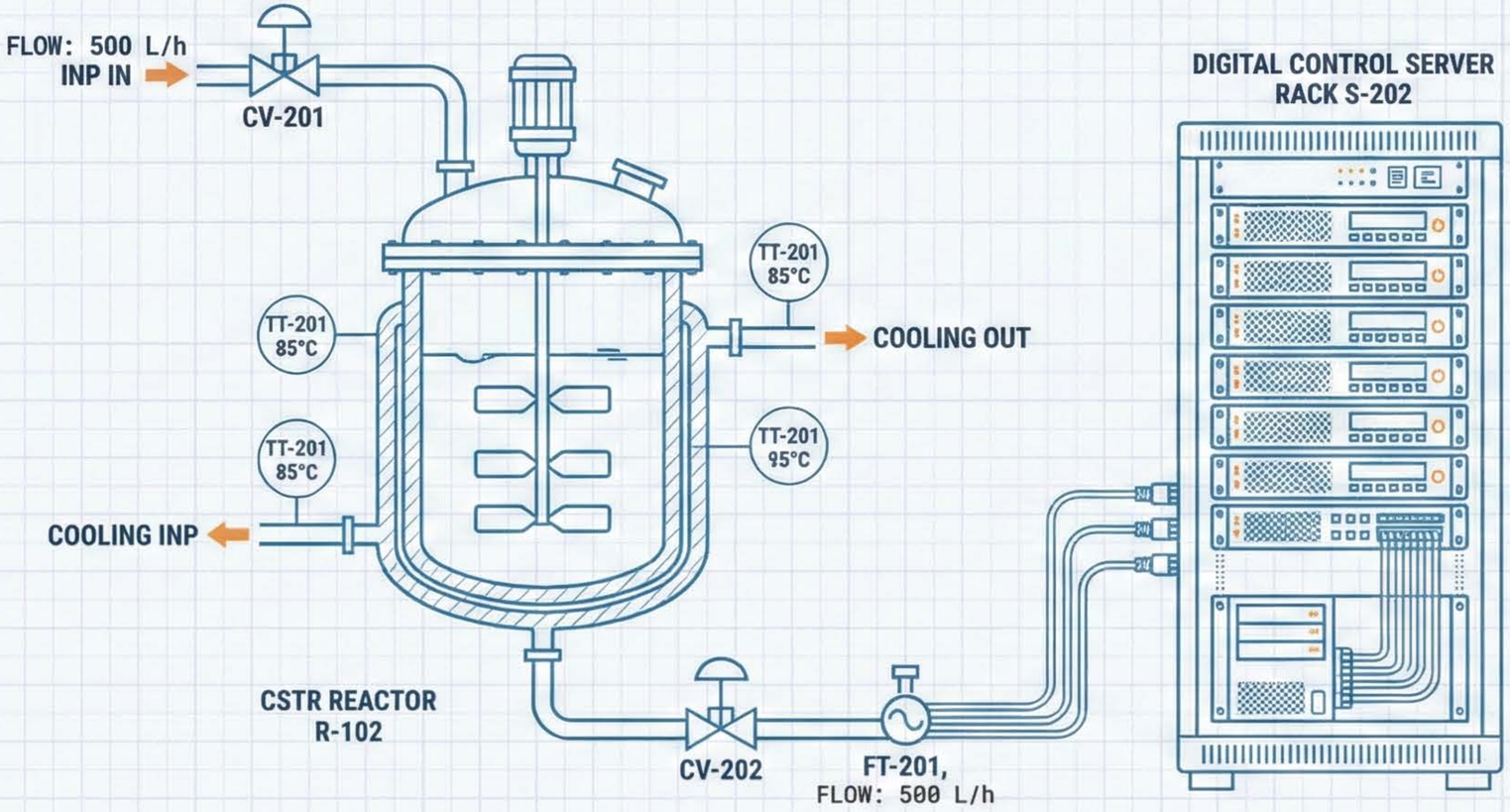
Unit 02 實務建議與課程總結

打造穩健的化工程式碼：從規範到實踐



NotebookLM

為什麼程式碼風格 (Style) 很重要？

“程式碼被閱讀的頻率遠高於被撰寫的頻率”

Good Code

```
# 計算反應速率
k_rate = 0.5
concentration = 2.0
rate = k_rate * (concentration ** 2)
```

Clean Indentation

Descriptive Variables

結構清晰，語法正確

Bad Code

```
x=0.5
y=2.0
z=x*(y**2)#calc
```

Missing Spaces

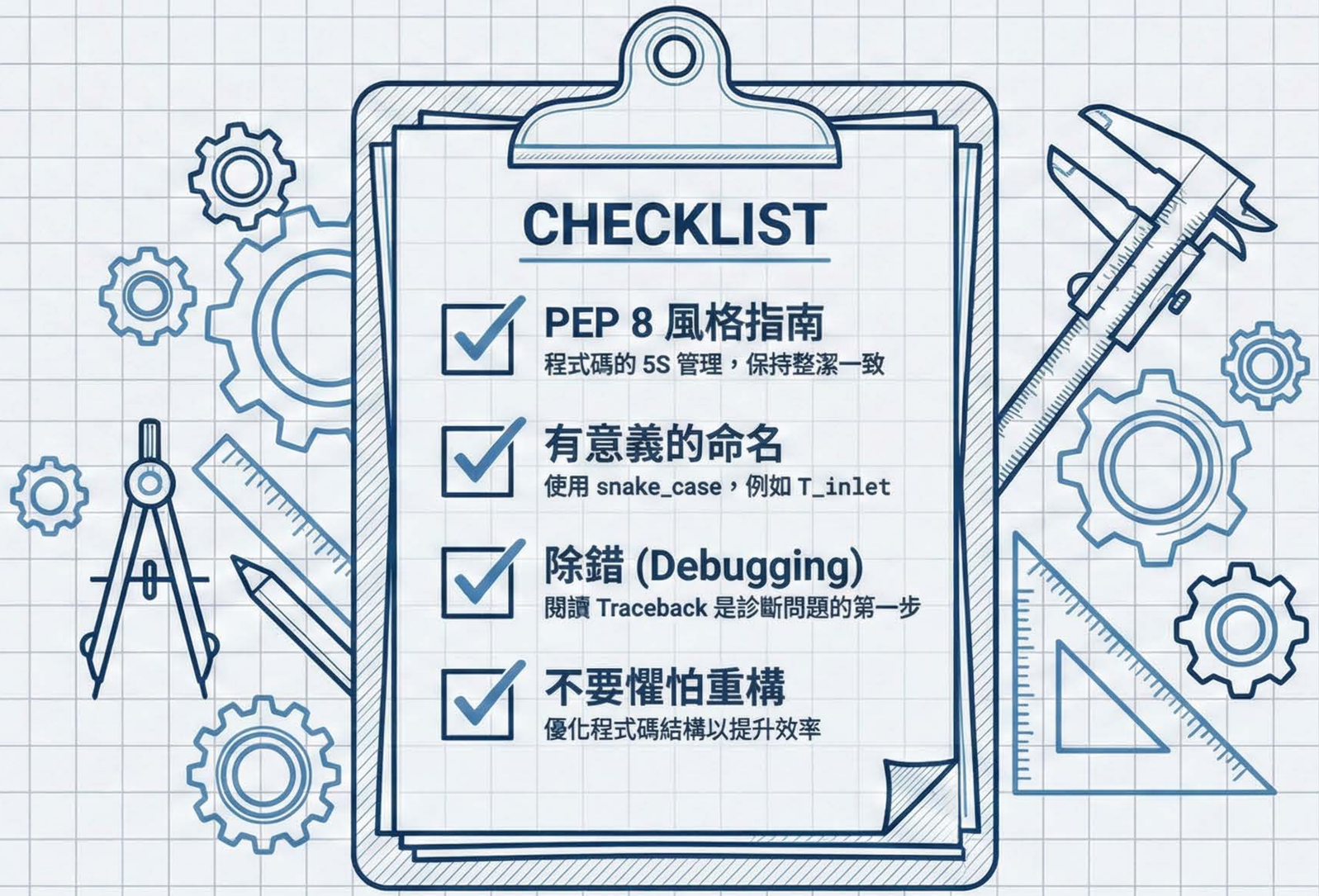
Unclear Names

Hard-to-Read

難以維護，易生 Bug

NotebookLM

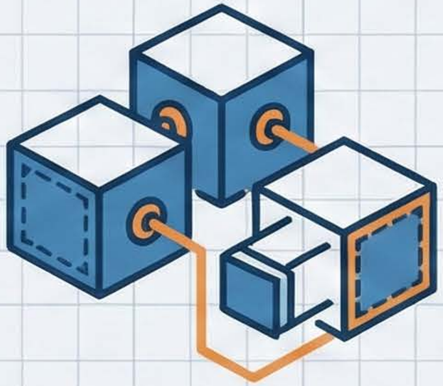
實務建議與最佳實踐 (Best Practices Checklist)



NotebookLM

程式碼組織與效能優化 (Organization & Optimization)

模組化 (Modularization)



模組化

將大問題拆解為小的函式 (Functions) 或模組。就像單元操作 (Unit Operations)，易於維護與測試。

效能優化 (Performance)

善用串列生成式

善用串列生成式 (List Comprehension)。語法更簡潔，執行效率更高。

```
[x**2 for x in range(10)]
```

除錯心態 (Diagnostic Mindset)

除錯心態

將除錯視為「解讀系統回饋」。Traceback 不是失敗，而是診斷報告。

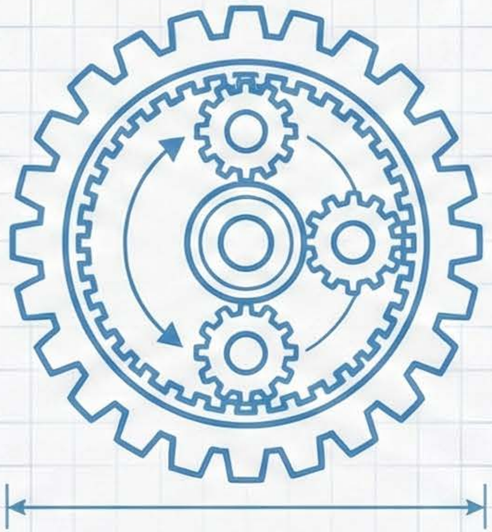
NotebookLM

系統盤點 (1)：原料與邏輯 (Inventory: Raw Materials & Logic)

變數與資料型態 (Raw Materials)

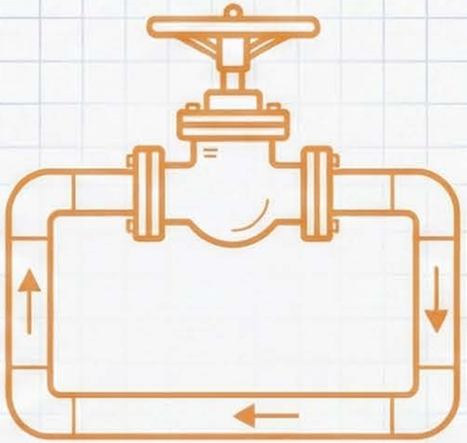
數值 (Numeric)
字串 (String)
串列 (List)
字典 (Dict)

運算子 (Operators)



算術 (+, -)
比較 (>, ==)
邏輯 (and, or)

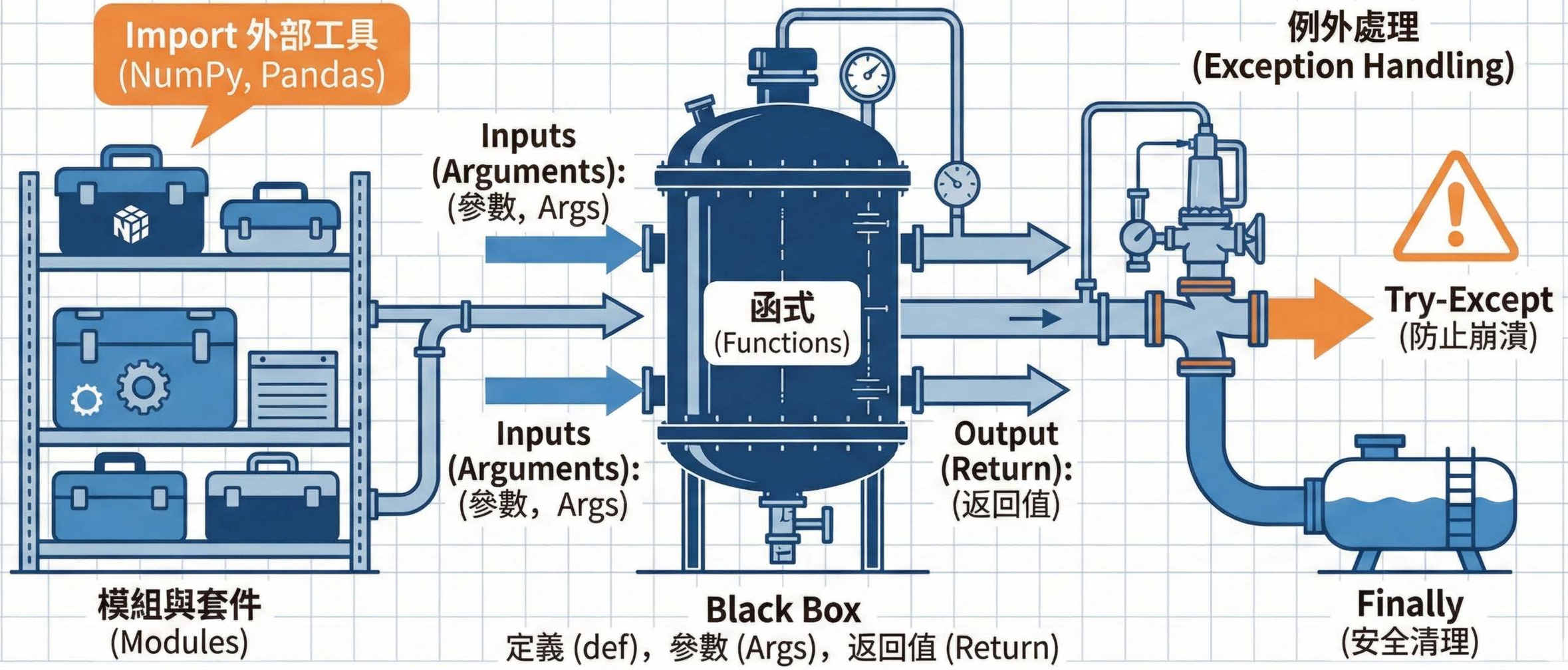
控制流程 (Piping & Logic)



條件語句 (If/Else) - 閥門
迴圈 (For/While) - 批次處理

NotebookLM

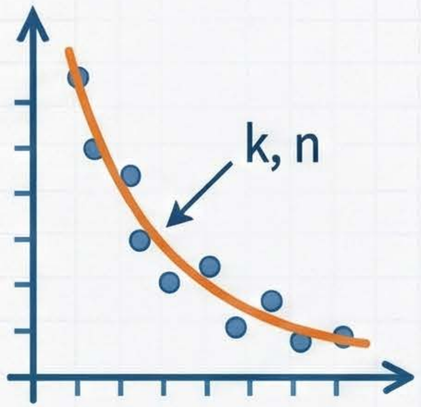
系統盤點 (2)：單元操作與安全系統
(Inventory: Operations & Safety)



為什麼選擇 Python？(The Strategic Advantage)

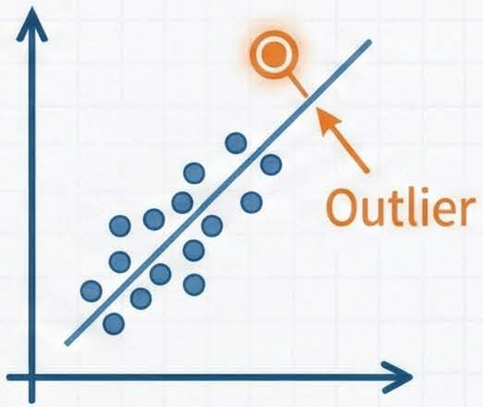
Python 不僅僅是程式語言，它是連接化工原理與人工智慧的通用語言。

反應動力學擬合 (Kinetics)



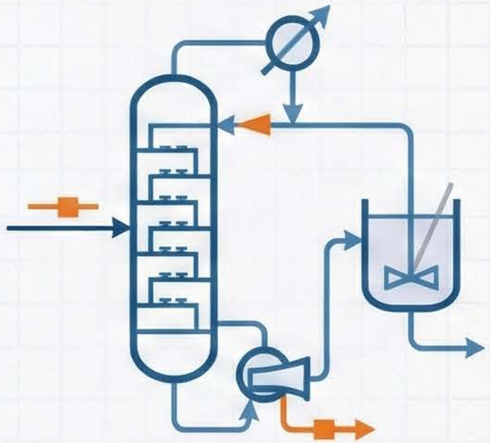
使用 SciPy 擬合實驗數據，求出反應速率常數 **k** 與級數 **n**。

異常檢測 (Anomaly Detection)



監控製程數據，自動識別偏離常態的**孤立點** (**Outliers**)。

單元操作模擬 (Simulation)



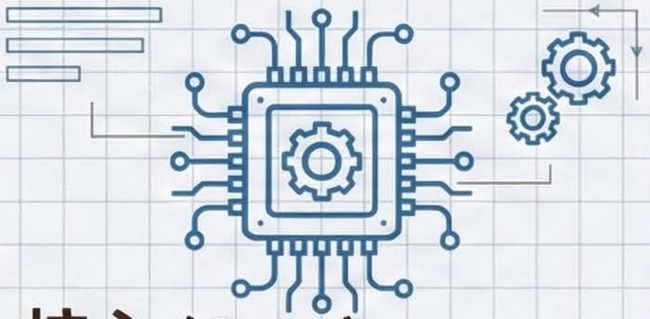
模擬**蒸餾塔**或 **CSTR** 在不同條件下的穩態輸出。

NotebookLM

結語：您的數位工具箱已備妥 (Conclusion)

橋樑 (Bridge)

Python 是連結化工原理與數據驅動決策的橋樑。



核心 (Core)

掌握 變數、迴圈 與 函式，即掌握了自動化的核心。

實踐 (Practice)

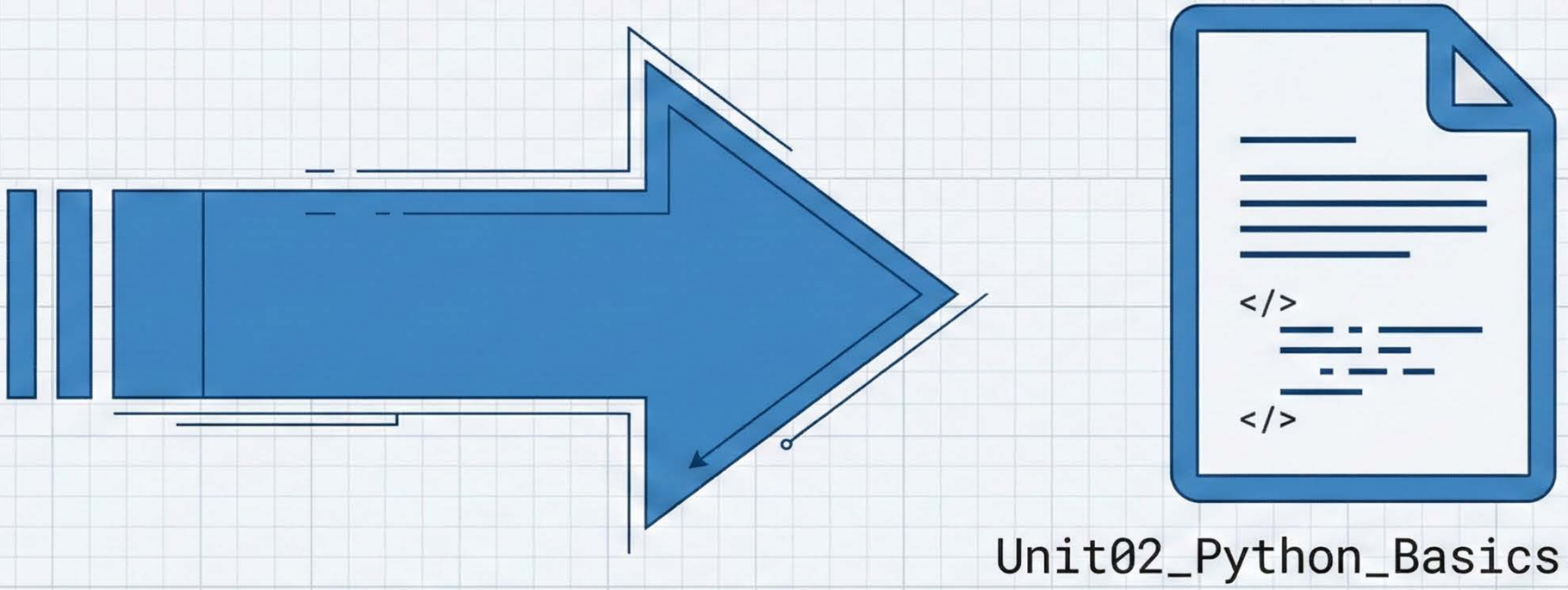
理論是不夠的，您需要親自寫 Code。

下一步 (Next Step)：

開啟 `Unit02_Python_Basics.ipynb`

現在就開始您的實作練習！

下一步 (Next Step)



Unit02_Python_Basics.ipynb

現在就開始您的實作練習！

SYSTEM ONLINE | Commissioning Complete. Handing over controls to Operator.

NotebookLM